

Go Gotchas & Advanced Concurrency Gotchas

Section 1: Common Go Gotchas

- Loop variable capture in goroutines.
 - Nil interface vs nil pointer confusion.
 - Defer executes in LIFO order.
 - Maps are not thread-safe.
 - Slice append may reallocate underlying array.
 - Nil map causes panic on write.
 - Range copies values (not references).
 - Structs containing slices cannot be compared.
 - Channel deadlock without receiver.
 - Goroutine leaks due to missing channel close.
 - Recover only works inside deferred functions.
 - Variable shadowing inside blocks.
 - Strings are immutable.
 - Unexported struct fields not marshaled in JSON.
 - Ignoring context in HTTP/DB calls causes leaks.
-

Section 2: Advanced Go Concurrency Gotchas (Production Level)

- Goroutine leaks due to missing context cancellation.
- Unbounded goroutines under high RPS.
- Channel blocking under heavy load.
- Ignoring context timeouts in DB/HTTP/Kafka.
- Mutex lock contention reducing scalability.
- RWMutex misuse under mixed workloads.
- Closing channel from receiver side.
- Sharing slices across goroutines without sync.
- Defer inside hot loops causing allocation overhead.
- Passing large structs by value (copy cost).
- Select without default leading to blocking.

- Using time.After in loop causing timer leaks.
- Heavy JSON marshalling in hot path increasing GC pressure.
- Not using pprof for production debugging.
- Kafka consumer spawning unbounded processing goroutines.